

KANA AI

Architektur-Handover · Stand 12. Mai 2026

Was ist KANA AI

KANA AI ist eine SaaS-Plattform, die spezialisierte KI-Agenten (gehostet als **Anthropic Managed Agents**) an Endkunden verkauft. Kunden registrieren sich, kaufen pro Stripe einen Agent-Zugang und können dann über eine eigene Chat-UI mit ihrem Agent arbeiten. Ziel ist es, Marketing-, Sales-, Support- und Research-Agenten als fertiges Produkt anzubieten — ohne dass Kunden selbst Modelle, APIs oder Tools konfigurieren müssen.

Aktueller Live-Stand

Live-URL	https://kanaai-49uy.vercel.app
GitHub-Repo	https://github.com/KLB2104/KANAAI
Aktiver Branch	main
Letzter Commit	d3dac40 — Fix Vercel build + Managed Agents API integration
Letzter Build	GRÜN — Deployment erfolgreich, Landing-Page redirected zu /sign-in

Architektur — die fünf Säulen

Die Plattform kombiniert fünf externe Services. Anthropic Managed Agents ist die **Zentrale** — die KI-Logik selbst läuft komplett bei Anthropic, KANA AI ist das Frontend + Zugangskontrolle + Abrechnung darum herum.

Next.js 15 + Vercel	Hosting und Server-Rendering der App. Auto-Deploy bei Git-Push auf main.
Clerk	Authentifizierung (Sign-Up, Sign-In, Sessions). Schützt alle Routen außer Landing, Sign-Up, Sign-In.
Supabase (Postgres)	Speichert wer welchen Agent gekauft hat (agent_access) und welche Sessions laufen (sessions).
Stripe	Bezahlung. Sendet Webhook nach erfolgreichem Kauf, der den Agent-Zugang in Supabase freischaltet.
Anthropic Managed Agents	Die eigentliche KI. Endpoints /v1/agents, /v1/environments, /v1/sessions, /v1/vaults hinter dem Browser.

Schlüsselwerte für die Plattform

Anthropic Environment ID	env_01RZfBf4sDSmLgvpRtD8eMZr
Test-Agent ID (Research)	agent_01G2ceoKDK99wXmYwBa6gVFQ
Anthropic Beta-Header	managed-agents-2026-04-01 (vom SDK auto-gesetzt ab v0.86.0)
SDK Version	@anthropic-ai/sdk ^0.95.0 (in package.json gepinnt)
Vercel Auto-Deploy	GitHub-Push auf main → automatischer Build

Datenfluss — wie ein Kauf zu Chat-Zugang wird

Der Lifecycle eines Kunden, von Registrierung bis Chat:

- 1. Kunde registriert sich über Clerk → bekommt `user_id` (Format `user_...`).
- 2. Kunde klickt Stripe-Checkout-Link → bezahlt → Stripe schickt POST an `/api/webhooks/stripe`.
- 3. Webhook prüft Signatur, liest `clerk_user_id` aus Metadata, findet den Agent über die Stripe-Product-ID.
- 4. Webhook schreibt Zeile in `agent_access` (Supabase) → Kunde hat Zugang.
- 5. Kunde öffnet Dashboard → Server liest alle Zugänge aus `agent_access` → zeigt Agent-Karten.
- 6. Klick auf eine Karte öffnet `/chat/[agentId]`.
- 7. Erste Nachricht → API-Route `/api/chat` → prüft Zugang → erstellt Anthropic-Session (Agent + Environment) → öffnet Event-Stream → sendet User-Event → streamt Antwort als Server-Sent-Events zurück ins UI.

Managed Agents — der korrekte API-Pfad

Das war der größte Bug im ursprünglichen Code: der falsche Pfad

`client.beta.managedAgents.sessions.*` existiert in keiner SDK-Version. Der echte Weg läuft über `client.beta.sessions.*`:

```
// 1) Session anlegen (Agent + Environment verbinden)
const session = await anthropic.beta.sessions.create({
  agent: agentId, // z.B. 'agent_01G2ceokDK99wXmYwBa6gVFQ'
  environment_id: environmentId, // z.B. 'env_01RZfBf4sDSmLgvpRtD8eMZr'
  title: 'Chat-Session',
});

// 2) Stream OPEN, dann SEND – Reihenfolge ist wichtig!
const stream = await anthropic.beta.sessions.events.stream(session.id);

await anthropic.beta.sessions.events.send(session.id, {
  events: [{
    type: 'user.message',
    content: [{ type: 'text', text: userMessage }],
  }],
});

// 3) Events lesen und ans Frontend pipen
for await (const event of stream) {
  if (event.type === 'agent.message.delta') {
    sendTextChunk(event.delta.text);
  } else if (event.type === 'session.status_idle') {
    break; // Agent ist fertig
  }
}
```

Steuerung über Drittsysteme — drei offizielle Wege

- **REST/SDK vom eigenen Server** — was die Plattform aktuell macht. Voller Kontrollfluss, gut für UI-Chat.
- **Inbound-Webhooks (`routines.fire`)** — externe Systeme (Stripe-Charge, Linear-Ticket, GitHub-CI) können Routinen anstoßen, ohne dass ein User im Chat tippt. Ideal für Automatisierung.

- **Outbound-Webhooks (Console → Settings → Webhooks)** — Anthropic ruft euren Server zurück bei session.started, session.idle, outcome.completed, vault.refresh_failed. So bleibt eure Datenbank live.

Was am Code repariert wurde

Ausgangslage: Initial-Build auf Vercel scheiterte, lokal ließ sich nicht zuverlässig testen. Diagnose und Fixes:

Datei	Status	Was wurde gemacht
package.json	NEU	SDK 0.39.0 → ^0.95.0 (frühere Versionen hatten keinen Managed-Agents-Support).
package-lock.json	NEU	Passend regeneriert; Vercel installiert deterministisch.
app/api/chat/route.ts	NEU	Komplett neu — beta.sessions.create + getrenntes events.stream + events.send mit environ
app/dashboard/page.tsx	NEU	Abmelde-Button → Clerk SignOutButton (vorher POST auf nicht existierende Route).
lib/supabase.ts	NEU	Clients lazy via Proxy — fehlende Env-Var crasht nicht mehr den Build-Prerender.
lib/agents.ts	NEU	getEnabledAgents() filtert PLACEHOLDER-IDs raus; nur echte Agents im UI.
.env.local.example	NEU	ANTHROPIC_ENVIRONMENT_ID dokumentiert.
ANLEITUNG.md	NEU	Schritt 5a (Environment in Console anlegen) ergänzt.
.gitignore	NEU	tsconfig.tsbuildinfo + *.log ignoriert.

Environment Variables — Pflichtliste

In Vercel **und** in der lokalen .env.local müssen alle folgenden gesetzt sein:

NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	dashboard.clerk.com → API Keys
CLERK_SECRET_KEY	dashboard.clerk.com → API Keys
NEXT_PUBLIC_CLERK_SIGN_IN_URL	/sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL	/sign-up
NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL	/dashboard
NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL	/dashboard
NEXT_PUBLIC_SUPABASE_URL	supabase.com → Settings → API
NEXT_PUBLIC_SUPABASE_ANON_KEY	supabase.com → Settings → API
SUPABASE_SERVICE_ROLE_KEY	supabase.com → Settings → API (geheim!)
ANTHROPIC_API_KEY	platform.claude.com → Settings → API Keys
ANTHROPIC_ENVIRONMENT_ID	env_01RZfBf4sDSmLgvpRtD8eMZr
STRIPE_SECRET_KEY	dashboard.stripe.com → API Keys
STRIPE_WEBHOOK_SECRET	wird beim Stripe-Webhook-Setup gesetzt (whsec_...)
NEXT_PUBLIC_APP_URL	https://kanaai-49uy.vercel.app

Smoke-Test (bevor wir Stripe scharfschalten)

Reihenfolge der Verifikationsschritte — Status zum Zeitpunkt des Handovers: **offen, vom User noch durchzuführen**.

- **1.** Sign-up auf <https://kanaai-49uy.vercel.app/sign-up> → User landet im leeren Dashboard.
- **2.** Supabase: Tabellen `agent_access` und `sessions` existieren. Falls nicht: `supabase-schema.sql` im SQL-Editor ausführen.
- **3.** Manuell INSERT in `agent_access` mit eigener Clerk-user_id und `agent_01G2ceoKDK99wXmYwBa6gVFQ`.
- **4.** Dashboard neu laden → Research-Agent-Karte erscheint.
- **5.** Karte klicken → Chat öffnen → 'Sag Hallo' senden. Drei mögliche Ausgänge: Antwort (gut), Fehlertext im Chat (`ANTHROPIC_ENVIRONMENT_ID` oder API-Key prüfen), oder 500 (Vercel-Logs → `/api/chat` lesen).

Nächste Etappe — Stripe-Integration (vom User priorisiert)

Sechs Schritte, in dieser Reihenfolge:

- **1. Produkt anlegen** — Stripe Dashboard → Products → Add product. Pricing wählen (einmalig oder Abo). Die `prod_...`-ID kopieren.
- **2. Product-ID in lib/agents.ts eintragen** — den `PLACEHOLDER_PROD_RESEARCH`-String durch die echte ID ersetzen. Erst dann verteilt der Stripe-Webhook auch Zugänge für diesen Agent.
- **3. Dynamischer Checkout** — der aktuelle Code im Dashboard hat einen Platzhalter-Link. Eine neue API-Route `/api/checkout` muss eine Stripe-Checkout-Session pro eingeloggtem User erstellen und `clerk_user_id` in die Metadata schreiben. Dieser Code-Patch ist noch nicht geschrieben — wäre der nächste Commit.
- **4. Webhook in Stripe registrieren** — Developers → Webhooks → URL <https://kanaai-49uy.vercel.app/api/webhooks/stripe>, Event `checkout.session.completed`.
- **5. Signing-Secret holen** — Stripe zeigt nach Anlage ein `whsec_...`-Secret, einmalig sichtbar. Sofort kopieren.
- **6. STRIPE_WEBHOOK_SECRET in Vercel setzen** und Redeploy auslösen (Vercel zieht Env Vars beim Build, nicht zur Runtime nach).

Roadmap danach

- **Echte Agents anlegen** — in platform.claude.com → Managed Agents → Agents je einen Sales- und Support-Agent bauen (mit System-Prompt, Tools, ggf. MCP-Servern), IDs ersetzen die `PLACEHOLDER_*`-Einträge.
- **Eigene Domain** — z.B. kanaai.com registrieren und in Vercel verknüpfen. Macht den Auftritt seriös und ist Voraussetzung für Stripe-Live-Modus.
- **Live-Modus aktivieren** — in Stripe vom Test- auf Live-Modus wechseln, Anthropic-Key auf Production-Key umstellen, Clerk Production-Instance verbinden.
- **Outbound-Webhooks von Anthropic** — für Operations-Monitoring (welche Sessions laufen, welche idle, welche Fehler), Logging in Supabase erweitern.

Bekannte Stolpersteine

- **Vercel deployt nicht automatisch** — kann passieren, wenn der GitHub-Hook deaktiviert ist. Workaround: Deployments-Tab → letztes Deployment → Redeploy ohne Build-Cache.
- **Env Vars fehlen beim Build** — typischer Fehler: 'Error: @clerk/nextjs: Missing publishableKey' beim Prerender. Lösung: alle Variablen aus der Pflichtliste in Vercel hinterlegen, dann Redeploy.
- **SDK ist zu alt** — beta.sessions.* gibt es erst ab @anthropic-ai/sdk 0.86.0. Wenn jemand die Version nach unten zieht, scheitert der Chat zur Laufzeit.
- **environment_id vergessen** — eine Session ist immer Agent + Environment. Fehlt die env_id, antwortet die API mit 400.
- **Lock-File asynchron** — wenn package.json geändert wird ohne npm install, ist package-lock.json out-of-sync. Vercel CI bricht dann ab. Lösung: npm install --package-lock-only und committen.

Promptvorlage für den nächsten Chat

Dieses Dokument als Datei beifügen und folgenden Prompt verwenden:

Hier ist ein Handover-PDF zu meinem Projekt KANA AI (SaaS-Plattform für Anthropic Managed Agents). Bitte lies es vollständig, fasse den Stand kurz zusammen und sag mir, womit wir weitermachen sollten. Mein aktuelles Ziel: [hier eintragen, z.B. Stripe-Checkout fertig bauen / Sales-Agent in Console anlegen / Custom Domain anbinden].

Erstellt für KLB · 12. Mai 2026 · Repo: github.com/KLB2104/KANAAI · Live: kanaai-49uy.vercel.app